Je code mon jeu

JavaScript est le langage exécuté par le navigateur. Dans `game.js`, tu écris les réglages et les règles de ton jeu.

1. Donner un nom à une information

```
const playerName = "Nova";
const speed = 4;
const gravity = 0.45;
```

`const` déclare une donnée. `playerName` est son nom ; `"Nova"` est sa valeur. Le texte a des guillemets ; un nombre n'en a pas.

4 est un nombre, `"4"` est du texte. Les décimaux utilisent un point : `0.45`.

2. Calculer la nouvelle position

`startX` est le départ. `player.x` est la position actuelle. Sur l'axe X, un nombre plus grand place le héros plus à droite.

```
player.x = player.x + speed;
```

On calcule d'abord à droite de =, puis on donne le résultat à `player.x`. Avec X = 50 et un pas de 4, la nouvelle position vaut 54.

Ajouter un pas fait aller à droite. Soustraire un pas fait aller à gauche.

3. Agir seulement si une condition est vraie

`if` veut dire « si ». Les accolades `{ }` entourent les actions de la condition.

```
function updateX(player, keys, speed) {
  if (keys.right) {
    player.x = player.x + speed;
  }
  if (keys.left) {
    player.x = player.x - speed;
  }
}
```

`function` regroupe des instructions. Le jeu appelle `updateX` régulièrement. `keys.right` et `keys.left` indiquent les touches pressées. Sans touche, aucun de ces calculs ne se fait.

4. Retrouver mes réglages

Réglage	Ce qu'il change	Valeurs de l'atelier
`startX`	La position de départ	24 à 70
`speed`	La taille d'un pas	3 à 5
`jumpPower`	La puissance du saut	9 à 11
`gravity`	La chute du héros	0.35 à 0.6
`fruitPoints`	Les points d'un fruit	1, 2 ou 3
`targetScore`	Le score à atteindre	3, 4 ou 5
`fruitGap`	L'écart entre les fruits	16 à 24

Je comprends et je teste

5. Une fonction donne une réponse avec return

Les paramètres sont les informations reçues. `return` renvoie la réponse.

```
function addPoints(score, points) {
  return score + points;
}
```

Ici, on garde l'ancien score et on ajoute les nouveaux points : 7 + 2 donne 9.

Dans une fonction, `return count * 2;` renvoie le double de `count` : * signifie multiplier. Pour 3 fruits, la prime vaut 6.

6. Vrai ou faux : les booléens

`true` veut dire vrai ; `false` veut dire faux. Ils s'écrivent sans guillemets.

```
function canJump(player) {
  return player.onGround;
}
```

`player.onGround` est vrai au sol, faux dans les airs : le saut n'est autorisé qu'au sol. `trap.active` indique si le feu est allumé.

Sur l'écran, Y augmente vers le bas. Une vitesse négative, comme `-jumpPower`, fait monter le héros.

7. Comparer et demander deux conditions

`>=` signifie « supérieur ou égal ». `3 >= 3` donne `true` ; `2 >= 3` donne `false`. Le signe `>` n'accepte pas l'égalité.

```
function canOpen(score, target, flag) {
  return score >= target && flag;
}
```

&& veut dire « et » : il faut assez de points ET le drapeau activé. Si l'une des deux conditions est fausse, l'arrivée reste fermée.

Attention : = donne une valeur ; >= compare deux valeurs.

8. Assembler du texte

```
const message = "Bravo " + playerName + " !";
```

Avec du texte, + assemble les morceaux. Si le nom est Nova, on obtient « Bravo Nova ! ». Avec des nombres, + additionne.

9. Tester et corriger

Prédis le résultat → change un réglage → clique sur Vérifier → joue → observe et explique ce qui a changé.

Garde ton code dans le même fichier. Avec `const`, déclare chaque nom une seule fois : pour changer un réglage, modifie sa ligne puis vérifie.

Si ça bloque : relis les majuscules, les guillemets, les parenthèses et les accolades. Vérifie aussi les points décimaux et les `;` à la fin des lignes de valeur ou de calcul.

Code valide ne veut pas toujours dire défi terminé : réalise aussi le saut, la récolte ou le trajet demandé. Pour comparer, change une chose à la fois.